

STOR 538

Predicting NFL Game Spread, Total Points, and Total Passing Yards using Machine Learning
Models

By Caleb Promislow, Joshua Loo, Diya Joshi, James Payette, Victor Mendoza-Alonso

Part 1: Data Gathering & Cleaning

We created three models to predict outcomes for NFL matches —game spread, total points, and total passing yards. We used data from *nflfastR*, along with **engineered variables**, to create our predictions. The data set used contained statistics from 2022 to 2025. The data cleaning process consisted of filtering to include only regular-season games to maintain a consistent training set for the model. **The columns used for the models included:** “game_id”, “season”, “week”, “home_team”, “away_team”, “posteam”, “total_home_score”, “total_away_score”, “play_type”, “epa”, and “passing_yard”. Finally, rows that were not informative, such as plays without an offensive team, missing EPA values, kickoffs, timeouts, and penalties, were dropped.

We created game-level statistics by grouping “game_id”, “season”, “week”, “home_team”, and “away_team”. **Then, we needed to create other metrics for our model.** Such as the maximum cumulative score for the home (“home_points”) and away (“away_points”) team. Total points variable, which is the sum of the home (“n_home_plays”) and away scores (“n_away_plays”). Number of plays, which counted the number of plays where the home or away team was on offense. Total EPA, which is the sum of the expected points added for home (“home_epa”) and away teams (“away_epa”). We engineered two variables that were the foundation for all three models: Elo ratings and EPA per play. Elo ratings quantify the quality and consistency of **each team**, based on their past performance. Each team starts with a baseline rating of 1500, and then after each game, the rating is updated by this formula:

$$R_{\text{New}} = R_{\text{old}} + K * M * (S - E)$$

Where R is the old rating. $K = 20$, a constant which controls rating volatility. M is the margin multiplier based on how much a team won or lost by. S is the game outcome where 1 equals a

win, 0 equals a loss, and 0.5 equals a tie. E is the expected win probability from both teams' pre-game ratings, which is adjusted by a home-field advantage of an additional 55 Elo points. Ratings are reset using a 0.75 decaying factor to account for changes like roster or coaching changes. **An Elo system is beneficial because it self-corrects the prediction, and it can adjust the rating of a team based on the opponent that the team is playing.** The other variable we engineered was EPA per play, which can be calculated by:

$$\text{Total EPA} / \text{Number of offensive plays}$$

EPA per play tells you how much value each offensive play adds to the score. Teams with high EPA per play are better at turning possessions into points. Elo and EPA per play combined work well to create a base for our models. Elo represents a long-term consistency, while **EPA per play** represents a short-term performance.

For all three models, we split the data into 80/20: 80 percent for testing and 20 percent for training. **Our dataset included 2025 data, and the prediction functions for all our models were trained to predict for games during weeks 7-11 using the latest Elo ratings and the most recent EPA per play passing statistics; therefore, our predictions reflect current trends.**

Part 2: Spread Variable

To predict the spread of games, we looked at three different models: least squares linear regression, random forest, and XGBoost. The least squares regression model was used as a baseline model for predictions and to look for linear interactions. Random Forest and XGBoost were used to examine more nonlinear interactions. For each model, we made sure to use the same variables. The variables we found to be the strongest were “home_elo_pre” (the home

team's elo before the game), "away_elo_pre" (the away team's elo before the game), and home_field (a value that the models that need an intercept can learn the intercept for home field advantage). "Home_field" was given a constant value of 1. Variables consisting of epa ("home_epa_per_play", "away_epa_per_play") were shown to be unimportant to predicting spread, as they would not increase/decrease MAE and RMSE, signifying that ELO and home field advantage were the major variables.

The first model that we looked at was linear regression (or least squares linear regression). We used linear regression as a starting point to quantify how each feature would affect spread within a linear context. It primarily focused on the relationship between the team's strengths and the point spread of that game.

The next model we examined was a random forest regressor. For this, we used 500 trees and standard sampling. The random forest model was trained using 5-fold repeated cross-validation to make sure that mtry and other parameters were generalized properly for unseen data. This number was used to try to keep a more robust test when predicting. The random forest focused on looking at the more nonlinear interactions that linear regression was unable to find.

The final model we examined was XGBoost. For this model, we had to tune various parameters for the final hyperparameter set. These were the final values that we ended up with:

- eta = 0.03,
- Max_depth = 4,
- subsample = 0.75,
- colsample_bytree = 0.75,
- nrounds = 300,

For each of these values, we tested with a range of different numbers to see which values would fit our dataset the best, while trying to avoid overfitting and underfitting. For eta, we started with 0.05. From there, we went with higher values, stopping at 0.1. However, those values started to show overfitting. After testing out higher values, we tested lower values. After 0.03, values, stopping at 0.01, would require more time and ended up being too slow when it came to predicting. For max_depth, we started at a value of 3, which immediately started showing signs of underfitting the dataset. As we went higher, any number that went past 4 started showing signs of overfitting, leaving 4 as the optimal value. We did similar methods with subsample and colsaple_bytree, where we would test to see which values provided the best results while properly fitting the data set, and we landed on 0.75.

The model that we chose as our best model was the XGBoost model. When comparing the MAE and RMSE of the test data (2022-2023 predicting 2024), we found that least squares linear regression gave the highest values for both metrics (table below). The second highest was given by random forest. XGBoost gave the lowest MAE and RMSE scores.

Model	MAE	RMSE
Linear Regression	10.04	13.35
Random Forest	9.83	13.33
XGBoost	9.81	13.23

To collect the predictions for future games regarding spread, we created a function that would take the home team, the away team, the latest ELO, and the model that we would be using. Since we are using XGBoost, the XGBoost model would then use the training data to predict the outcomes. We would then read a CSV file that contained all of the matches that we are predicting. From there, we made a “for loop” that cycled through the CSV file, which would

take the home team name and the away team name and input them into the function that we created to predict. It would then put all the predictions into a dataframe with columns being labeled for the week, the home team, the away team, and the spread. We had included columns for total points and passing yards, but they were filled with NA to be later filled with values after each model was finished predicting.

Part 3: Total Points

Our group experimented on a variety of models to best predict the Total Points that would be scored in a given NFL game. Similar to how we approached the point spread, we tested an ordinary least squares linear regression model, XGBoost, and a random forest. For each of these models, we used the same predictors of “home_elo_pre”, “away_elo_pre”, along with “home_epa_pp” and “away_epa_pp” (the expected points added per play of the home and away team).

The elo's helped the models by showing how good/strong each team is before the game. While this is naturally useful in determining which team wins a game, knowing the quality of teams involved in games was something we also hypothesized would be directly involved in the score of the game. If one team's elo is much higher than another for example, it could indicate a mismatched game where the team with higher elo dominates and controls the scoring and thus the total points in the game. Home and away expected points added per play are also very useful predictors because a team that is expected to score more points per play is likely to contribute more to the total game score. Measuring the epa of home and away teams, allows the models to find games where both offenses look strong, weak, or somewhere in between before using that information to best predict the total points.

The first model we tested was the OLS model. Even though this was the simplest of the models, it produced an impressive MAE of 8.28 and RMSE of 10.25 (as shown in the table below) on the test set.

The next model we tested for total points was the random forest model with 500 trees and 3 features per split (mtry of 3). Contrary to the outcomes for the spread variable, this time the random forest model actually underperformed the least squares linear regression model with an MAE of 8.62 and a RMSE of 10.63. While this is only slightly worse, it was enough for us to rule random forest out of use as our predictive model.

We did our third test on the XGBoost model. After messing around with the parameters for a little while, we finally decided on the following ones:

Eta = .03

Max_depth = 3

Subsample = 0.8

Colsample_bytree = 0.8

Nrounds = 300

Total points in the NFL are generally difficult to predict and have lots of variance. One can just look at the Jets' games from this season to see that. As such, a smaller eta of .03 makes our model less likely to overfit to individual games. Combining that smaller eta with an nrounds of 300 allows the model to create more controlled predictions that don't change rapidly or overfit, but still have lots of rounds of information to go over.

Similar reasoning is used for keeping the "max_depth" to 3. The number of variables is relatively small, and we don't want to overfit by having too many splits on the trees. We did

various experiments on subsample and colsample_bytree, where we tested different values between .7 - .85 before determining that .8 gave the best results without overfitting the data.

Despite all the messing around with the XGBoost model and working to find the best parameters possible, it still ended up having a worse MAE and RMSE than the linear regression did (8.32 and 10.43, respectively), which came as a big surprise. As such, our group decided to use the linear regression model as our predictor model for total points. The MAE and RMSE results for each model are shown in the table below.

Model	MAE	RMSE
Linear Regression	8.28	10.25
Random Forest	8.62	10.63
XGBoost	8.32	10.43

The linear model equation was a function of only the four variables: home_elo, away_elo, home_epa_pp, and away_epa_pp. The equation for the linear model ended up being as so:

$$\text{TotalPoints} = 31.356 + (.0154 * \text{home_elo}) - (.006 * \text{away_elo}) + (35.911 * \text{home_epa_pp}) + (33.215 * \text{away_epa_pp}).$$

Home_elo was significant at the 5% level, and home_epa_pp and away_epa_pp both had extremely low p-values and were significant at the 1% level each. While away_elo was not statistically significant on its own, it still helps the model account for the strength of the away team and improves overall model accuracy, so we naturally kept it.

Part 4: Passing Yards

To predict total passing yards for individual NFL games, our group tested two different modeling approaches. We first used a multiple linear regression before settling on an XGBoost regression. The linear model served as a baseline that helped identify straightforward relationships between offensive and defensive metrics with overall passing production, while XGBoost was implemented as a more advanced, nonlinear model that was capable of capturing complex interactions between features. Both models utilized play-by-play information in *nflfastR* that was used to derive defensive and offensive performance statistics, pressure rate, passing and rushing efficiency (EPA), as well as ELO-based strength ratings.

The multiple linear regression model was used primarily to quantify the influence of offensive and defensive metrics in a simple linear context. The dependent variable was total passing yards per game, while the independent variables included team-level passing and rushing EPA, total points scored for both teams, total pass attempts, and pressure rate. If a defense is able to generate pressure consistently, the opposing offense would then have a more difficult time passing consistently. Rushing efficiency for an offense would also logically impact passing, as a more effective rushing game allows the offense to take a more balanced offensive approach, while a weak running game will make the offense dependent on the pass. Then the other variables could be easily assumed to directly impact passing, as pass efficiency, attempts, and points scored would obviously impact the success of a passing game and total passing yards for a team. The linear regression was useful for interpretability as it allowed us to see which variables were statistically significant and how each variable influenced total passing production. Although the model was somewhat limited by the assumption of linearity, our residual analysis

suggested that several relationships between features were nonlinear, which created the need for a more flexible modeling approach.

As a result of this, we developed an XGBoost regression model as our final predictive model. This method builds decision trees sequentially, where each tree corrects the errors of the previous one. It is highly effective for structured tabular data like ours and is able to handle nonlinear feature interactions automatically. For our implementation, we turned several hyperparameters that would balance predictive accuracy and generalization:

- `eta = 0.05`
- `max_depth = 5`
- `subsample = 0.8`
- `colsample_bytree = 0.8`
- `nrounds = 300`

We chose these parameters after iterative testing to minimize overfitting and ensure stable performance across training and validation data. Shallower trees with a `max_depth` level of 3-4 underfit the data, while deeper trees captured too much, so we found 5 to be the optimal balance.

When we compared the model performances, the XGBoost model clearly outperformed the linear regression for passing yards across both the Mean Absolute Error and the Root Mean Square Error metrics. Despite this, the linear regression model was still useful in providing a good baseline in understanding the directional effects of specific variables on total passing yards, but lacked the flexibility to capture certain interaction effects. The model likely could have still yielded somewhat decent predictions, but we felt that the XGBoost model would overall be more

reliable. XGBoost's ability to incorporate nonlinear relationships helped lead to more accurate and robust predictions, especially in games where one team's offensive and defensive strengths were significantly unbalanced. Our group ultimately decided to utilize the XGBoost model as our final predictive tool for predicting passing yards due to its superior adaptability and accuracy. It reduced prediction error compared to the linear model, but also better modeled the dynamic and nonlinear nature of football performance data.